```
from asgiref.sync import async_to_sync
from django.utils import timezone

from core.models import Employee, PresenceLog, ActivityTracker,
ReminderLog
from core.services.graph import get_user_presence
from core.services.teams_sender import send_proactive_message


WORKING_STATUSES = [
    "Available",
    "Busy",
    "InACall",
    "InAMeeting",
]

GOOD_MORNING_ALLOWED_STATUSES = [
    "Available",
    "Busy",
    "InACall",
    "InAMeeting",
]

CHECK_INTERVAL_MINUTES = 2

WELLNESS_REMINDER_MESSAGES = [
```

```
    "💧 Hydration check! Take a sip of water and relax your shoulders.",
    "🧘 Short break time! Stand up, stretch, and refresh your body for 2 minutes.",
    "👀 Give your eyes a break — look away from the screen for a few moments.",
    "🚶 Time to move a little! A short walk can help reset your focus.",
    "🦁 Stretch reminder! Roll your neck and shoulders gently to reduce stiffness.",
    "🌿 Pause for a moment — breathe deeply and reset your energy.",
    "🪑 Sitting too long? Stand up and loosen your back and legs for a minute.",
    "🔬 Wellness check! Drink water and take a mindful pause before continuing.",
    "⏰ Time for a quick wellness break! Please drink some water and stretch a little.",
    "🖥️ Screen break time — rest your eyes and blink a few times slowly.",
    "🧍 Quick posture check! Sit tall or stand up and stretch for a moment.",
    "🌬️ Take a deep breath — a short breathing break can improve focus.",
    "🚰 Friendly reminder: hydrate, stretch, and return refreshed.",
    "✨ Small break, big benefit — take 2 minutes to move and recharge.",
]


def is_within_working_hours():
    """
```

```
    Monday-Friday, 8:00 AM to 7:00 PM (Asia/Kolkata)
    """
    now = timezone.localtime()
    weekday = now.weekday()  # Monday=0, Sunday=6
    current_hour = now.hour

    is_weekday = weekday < 5
    within_hours = 8 <= current_hour < 19

    return is_weekday and within_hours


def get_next_reminder_message(employee):
    """
    Return a reminder message that has not yet been used today for this
employee.
    If all messages have been used today, fallback to first message.
    """
    today = timezone.localdate()

    used_messages_today = set(
        ReminderLog.objects.filter(
            employee=employee,
            sent_at__date=today
        ).values_list("message", flat=True)
    )

    for message in WELLNESS_REMINDER_MESSAGES:
        if message not in used_messages_today:
```

```
            return message

    # Fallback if all messages used in same day (unlikely but safe)
    return WELLNESS_REMINDER_MESSAGES[0]


def has_teams_chat_initialized(employee):
    """
    Check whether employee has opened the bot at least once and
conversation details exist.
    """
    return (
        bool(employee.teams_user_id)
        and bool(employee.teams_conversation_id)
        and bool(employee.teams_service_url)
    )


def fetch_employee_presence(employee, save_log=True):
    """
    Fetch employee presence safely.
    Returns status string or None if fetch fails.

    If save_log=True, also stores PresenceLog.
    """
    try:
        presence_data = get_user_presence(employee.azure_user_id)
        status = presence_data.get("availability")
```

```
        print(f"{employee.display_name} status: {status}")

        if save_log:
            PresenceLog.objects.create(
                employee=employee,
                status=status or "Unknown"
            )

        return status

    except Exception as e:
        print(f"Presence fetch failed for {employee.display_name}: {e}")
        return None


def check_employee_presence():
    """
    Runs every 2 minutes during working hours.
    Sends reminder after continuous working duration reaches employee
threshold.
    """
    if not is_within_working_hours():
        print("Outside working hours. Skipping presence check.")
        return

    employees = Employee.objects.filter(
        is_active=True,
        reminder_enabled=True,
    )
```

```
    for employee in employees:
        # User must have initialized chat at least once
        if not has_teams_chat_initialized(employee):
            print(f"Skipping {employee.display_name}: no Teams
conversation details yet")
            continue

        status = fetch_employee_presence(employee, save_log=True)

        if status is None:
            continue

        tracker, _ =
ActivityTracker.objects.get_or_create(employee=employee)

        if status in WORKING_STATUSES:
            tracker.consecutive_available_count += 1
            print(f"Working detected ->
{tracker.consecutive_available_count}")
        else:
            print(f"Resetting counter for {employee.display_name}")
            tracker.consecutive_available_count = 0

        # Example: 60 mins / 2 mins = 30 checks
        required_checks = max(
            1,
            employee.reminder_interval_minutes //
CHECK_INTERVAL_MINUTES
```

```
        )

        if tracker.consecutive_available_count >= required_checks:
            reminder_message = get_next_reminder_message(employee)

            ReminderLog.objects.create(
                employee=employee,
                message=reminder_message
            )

            print(f"Reminder triggered for {employee.display_name}")
            print(f"Reminder message: {reminder_message}")

            try:
                async_to_sync(send_proactive_message)(employee,
reminder_message)
                print(f"Reminder sent to Teams for
{employee.display_name}")
            except Exception as e:
                print(f"Failed to send Teams reminder to
{employee.display_name}: {e}")

            # Reset after reminder so next reminder comes after another
full interval
            tracker.consecutive_available_count = 0

        tracker.last_status = status
        tracker.save()
```

```
def send_good_morning_messages():
    """
    Sends Good Morning message only to active employees who:
    - have initialized Teams chat
    - are currently online/working (not offline/away)

    Runs at scheduled time Mon-Fri.
    """
    now = timezone.localtime()
    weekday = now.weekday()

    if weekday >= 5:
        print("Weekend. Skipping good morning messages.")
        return

    employees = Employee.objects.filter(
        is_active=True,
        reminder_enabled=True,
    )

    message = (
        "🌞 Good morning! Wishing you a productive and healthy
workday. "
        "I'll gently remind you to take wellness breaks during the day."
    )

    for employee in employees:
        if not has_teams_chat_initialized(employee):
```

```
            print(f"Skipping good morning for {employee.display_name}: no
Teams chat initialized")
            continue

        # Fetch live presence before sending Good Morning
        status = fetch_employee_presence(employee, save_log=True)

        if status is None:
            print(f"Skipping good morning for {employee.display_name}:
presence unavailable")
            continue

        if status not in GOOD_MORNING_ALLOWED_STATUSES:
            print(
                f"Skipping good morning for {employee.display_name}: "
                f"status is {status} (not eligible)"
            )
            continue

        try:
            async_to_sync(send_proactive_message)(employee, message)
            print(f"Good morning message sent to
{employee.display_name}")
        except Exception as e:
            print(f"Failed to send good morning to
{employee.display_name}: {e}")


def reset_daily_trackers():
```

```
    """
    Reset all activity counters before workday starts.
    Also delete all old PresenceLogs so only today's logs remain.
    Runs at 7:55 AM Mon-Fri.
    """
    # Delete old presence logs first
    today = timezone.localdate()
    deleted_count, _ = PresenceLog.objects.exclude(
        checked_at__date=today
    ).delete()

    # Reset trackers
    updated = ActivityTracker.objects.all().update(
        consecutive_available_count=0,
        last_status=None,
        reminder_sent=False,
    )

    print(f"Deleted old PresenceLog records: {deleted_count}")
    print(f"Reset {updated} activity trackers")
```

this is my presense_engine.py

```
from django.contrib import admin
from core.models import Employee, PresenceLog, ActivityTracker,
ReminderLog


@admin.register(Employee)
class EmployeeAdmin(admin.ModelAdmin):
    list_display = (
```

```
        "display_name",
        "email",
        "is_active",
        "reminder_enabled",
        "reminder_interval_minutes",
    )
    search_fields = ("display_name", "email")
    list_filter = ("is_active", "reminder_enabled")


@admin.register(PresenceLog)
class PresenceLogAdmin(admin.ModelAdmin):
    list_display = ("employee", "status", "checked_at")
    search_fields = ("employee__display_name",)
    list_filter = ("status", "checked_at")


@admin.register(ActivityTracker)
class ActivityTrackerAdmin(admin.ModelAdmin):
    list_display = ("employee", "consecutive_available_count",
"last_status")
    search_fields = ("employee__display_name",)


@admin.register(ReminderLog)
class ReminderLogAdmin(admin.ModelAdmin):
    list_display = ("employee", "sent_at", "message")
    search_fields = ("employee__display_name", "message")
    list_filter = ("sent_at",)
```

this is admin.py
when we send good morning message to employeese i want to generate morning log same a sreminder log which is once a day and i aslso want to show it in admin and also give me 10-12 unique good morning messages send every user diffrent good morning messages tell me now in detail what changes do i need to do proparlly

Perfect — this is the **right next improvement** for your project 👍

You want **3 things**:

1. **When Good Morning message is sent**, create a **GoodMorningLog** (just like `ReminderLog`)
2. Show that log in **Django Admin**
3. Have **10–12 unique Good Morning messages**, and send a **different message each day per employee** (avoid repeating same message on the same day / rotate nicely)

✅ Best approach (recommended)

Create a **new model**:

- `GoodMorningLog`
- store:
  - `employee`
  - `message`
  - `sent_at`

Then:

- add it to `admin.py`

- in `send_good_morning_messages()`:
  - choose a unique GM message for that employee for today
  - send Teams message
  - if success → create `GoodMorningLog`

This is the **cleanest design** because:

- `ReminderLog` = wellness reminders during the day
- `GoodMorningLog` = one morning greeting per day
- easier to track in admin
- no confusion in reports later

✅ Step-by-step changes

---

1) Update `models.py` → add `GoodMorningLog`

Open your `core/models.py`

Add this new model **below** `ReminderLog` (or anywhere in models file):

```
from django.db import models


class GoodMorningLog(models.Model):
    employee = models.ForeignKey(
        "Employee",
        on_delete=models.CASCADE,
```

```
        related_name="good_morning_logs"
    )
    message = models.TextField()
    sent_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-sent_at"]
        verbose_name = "Good Morning Log"
        verbose_name_plural = "Good Morning Logs"

    def __str__(self):
        return f"{self.employee.display_name} - {self.sent_at:%Y-%m-%d %H:%M}"
```

✅ Why this model?

It behaves exactly like `ReminderLog`, but specifically for Good Morning messages.

That means:

- **1 row per successful Good Morning send**
- you can check:
  - who got message
  - what exact message
  - when it was sent

---

2) Create migration and migrate

Run:

```bash
python manage.py makemigrations
```

Bash

This will create the new table in DB.

---

3) Update `admin.py` to show Good Morning logs

Your current `admin.py` imports:

```python
```

Python Run

Change it to:

```python
from django.contrib import admin
```

Python Run

Now add this admin registration **below ReminderLogAdmin**:

```python
@admin.register(GoodMorningLog)
class GoodMorningLogAdmin(admin.ModelAdmin):
```

```python
    list_display = ("employee", "sent_at", "message")
    search_fields = ("employee__display_name", "message")
```

Python Run

✅ Final `admin.py` should look like this

```python
from django.contrib import admin
from core.models import Employee, PresenceLog, ActivityTracker, ReminderLog, GoodMorningLog


@admin.register(Employee)
class EmployeeAdmin(admin.ModelAdmin):
    list_display = (
        "display_name",
        "email",
        "is_active",
        "reminder_enabled",
        "reminder_interval_minutes",
    )
    search_fields = ("display_name", "email")
    list_filter = ("is_active", "reminder_enabled")


@admin.register(PresenceLog)
class PresenceLogAdmin(admin.ModelAdmin):
    list_display = ("employee", "status", "checked_at")
    search_fields = ("employee__display_name",)
    list_filter = ("status", "checked_at")
```

```python
@admin.register(ActivityTracker)
class ActivityTrackerAdmin(admin.ModelAdmin):
    list_display = ("employee", "consecutive_available_count", "last_status")
    search_fields = ("employee__display_name",)


@admin.register(ReminderLog)
class ReminderLogAdmin(admin.ModelAdmin):
    list_display = ("employee", "sent_at", "message")
    search_fields = ("employee__display_name", "message")
    list_filter = ("sent_at",)


@admin.register(GoodMorningLog)
class GoodMorningLogAdmin(admin.ModelAdmin):
    list_display = ("employee", "sent_at", "message")
    search_fields = ("employee__display_name", "message")
    list_filter = ("sent_at",)
```

---

4) Update `presence_engine.py` imports

Currently you have:

Python Run

Change it to:

Python Run

---

5) Add 10–12 unique Good Morning messages

In `presence_engine.py`, below `WELLNESS_REMINDER_MESSAGES`, add:

```
GOOD_MORNING_MESSAGES = [
    "🌞 Good morning! Wishing you a productive and healthy workday. I’ll gently remind you to tak
    "🌻 Good morning! Hope you start your day with energy, focus, and a healthy routine.",
    "🌸 Good morning! Wishing you a calm mind, good posture, and a productive day ahead.",
    "💧 Good morning! Stay hydrated, take mindful breaks, and have a wonderful workday.",
    "🌿 Good morning! A fresh start for a fresh day — work well and remember to take care of your
    "✨ Good morning! Small healthy habits today can make a big difference — have a great day ahe
    "🧘 Good morning! Begin your day with focus and balance. I’ll be here to remind you to take s
    "🚶 Good morning! Wishing you a positive day filled with productivity and healthy movement.",
    "🌺 Good morning! Stay active, stretch often, and make today a great one.",
    "🌈 Good morning! Hope your day is productive, peaceful, and full of positive energy.",
    "💼 Good morning! Let’s make today productive and healthy — don’t forget to stretch and hydra
    "😊 Good morning! Wishing you a smooth workday with good focus, good health, and good energy.
]
```

---

6) Add helper function to pick a unique Good Morning message

Just like you already did for reminders, create a similar function.

Add this in `presence_engine.py` (below `get_next_reminder_message()` is a good place):

```python
def get_next_good_morning_message(employee):
    """
    Return a good morning message that has not yet been used today for this employee.
    If a message was already sent today, return None (to ensure only one per day).
    If all messages were used historically and you want rotation, fallback safely.
    """
    today = timezone.localdate()

    # If already sent today, do not send again
    already_sent_today = GoodMorningLog.objects.filter(
        employee=employee,
        sent_at__date=today
    ).exists()

    if already_sent_today:
        return None

    # Optional: avoid repeating recent messages by checking all historical GM logs
    used_messages = set(
        GoodMorningLog.objects.filter(
            employee=employee
        ).values_list("message", flat=True)
    )

    for message in GOOD_MORNING_MESSAGES:
        if message not in used_messages:
            return message

    # If all messages have been used historically, rotate from beginning
    # You can also randomize later if you want
    total_sent = GoodMorningLog.objects.filter(employee=employee).count()
    return GOOD_MORNING_MESSAGES[total_sent % len(GOOD_MORNING_MESSAGES)]
```

✅ What this function does

For each employee:

- If already got Good Morning **today** → returns `None`
- Else:
  - tries to send a message they have **never received before**
- If all 12 messages are already used historically:
  - it rotates based on count
  - so it still stays nicely distributed

So this gives you:

- **only 1 GM per day**
- **different message every day**
- **each employee gets different sequence based on their own history**

---

7) Update `send_good_morning_messages()` properly

Now we'll improve your function.

❌ Current issue in your function

Right now:

- it sends **same message to all employees**
- it does **not create GoodMorningLog**
- it may send again if scheduler accidentally runs twice (rare but possible)

✅ Replace your current `send_good_morning_messages()` with this full version

```
def send_good_morning_messages():
    """
    Sends Good Morning message only to active employees who:
    - have initialized Teams chat
    - are currently online/working (not offline/away)
    - have NOT already received Good Morning today

    Runs at scheduled time Mon-Fri.
    """
    now = timezone.localtime()
    weekday = now.weekday()

    if weekday >= 5:
        print("Weekend. Skipping good morning messages.")
        return

    employees = Employee.objects.filter(
        is_active=True,
        reminder_enabled=True,
    )

    for employee in employees:
        if not has_teams_chat_initialized(employee):
            print(f"Skipping good morning for {employee.display_name}: no Teams chat initialized"
            continue

        # Prevent duplicate Good Morning in same day
        good_morning_message = get_next_good_morning_message(employee)
        if good_morning_message is None:
```

```
            print(f"Skipping good morning for {employee.display_name}: already sent today")
            continue

        # Fetch live presence before sending Good Morning
        status = fetch_employee_presence(employee, save_log=True)

        if status is None:
            print(f"Skipping good morning for {employee.display_name}: presence unavailable")
            continue

        if status not in GOOD_MORNING_ALLOWED_STATUSES:
            print(
                f"Skipping good morning for {employee.display_name}: "
                f"status is {status} (not eligible)"
            )
            continue

        try:
            async_to_sync(send_proactive_message)(employee, good_morning_message)
            print(f"Good morning message sent to {employee.display_name}")
            print(f"Good morning message: {good_morning_message}")

            # Create log only after successful send
            GoodMorningLog.objects.create(
                employee=employee,
                message=good_morning_message
            )

            print(f"Good morning log created for {employee.display_name}")

        except Exception as e:
            print(f"Failed to send good morning to {employee.display_name}: {e}")
```

8) Important behavior after this change

Now your Good Morning flow will be:

For each employee:

1. Check weekend?
2. Check active + reminder enabled
3. Check Teams chat initialized
4. Check if Good Morning already sent **today**
   - if yes → skip
5. Fetch live presence
6. Only allow if:
   - `Available`
   - `Busy`
   - `InACall`
   - `InAMeeting`
7. Send message
8. If send successful:
   - create `GoodMorningLog`

---

9) Very important note about PresenceLog

You already have:

Python Run

inside `send_good_morning_messages()`

That means:

- every morning when GM runs,
- one **PresenceLog** entry is created

So after this change, at Good Morning time you will get:

- **1 PresenceLog**
- **1 GoodMorningLog** (if message sent)

This is **totally fine** and actually good for audit.

---

10) Do you need to reset GoodMorningLog daily?

👉 **No. Do NOT delete GoodMorningLog daily.**

Why?

Because it is useful history.

You should keep:

- `ReminderLog` → keep history
- `GoodMorningLog` → keep history
- `PresenceLog` → okay to delete old logs daily (because it becomes too large fast)

So your `reset_daily_trackers()` can stay as is.

---

11) Optional improvement (recommended): show last Good Morning in Employee admin

This is optional, but very useful.

If you want, in `EmployeeAdmin`, we can add:

- last Good Morning sent time

Update `EmployeeAdmin`:

```
@admin.register(Employee)
class EmployeeAdmin(admin.ModelAdmin):
    list_display = (
        "display_name",
        "email",
        "is_active",
        "reminder_enabled",
        "reminder_interval_minutes",
        "last_good_morning_sent",
    )
    search_fields = ("display_name", "email")
    list_filter = ("is_active", "reminder_enabled")

    def last_good_morning_sent(self, obj):
        last_log = obj.good_morning_logs.order_by("-sent_at").first()
        return last_log.sent_at if last_log else "-"
    last_good_morning_sent.short_description = "Last Good Morning"
```

This works because in model we used:

Python Run

---

12) Optional improvement (very important for duplicate protection)

If APScheduler accidentally runs `send_good_morning_messages()` twice around same time (rare but possible), your current check is usually enough.

But for **strong duplicate protection**, you can also add a **database constraint** later (advanced).

For now, your current logic:

```python
already_sent_today = GoodMorningLog.objects.filter(
    employee=employee,
    sent_at__date=today
```

Python Run

is enough for local + single scheduler instance.

Since you are currently working on scheduler stability in Django, this is the **best practical approach right now**.

---

13) Final recommended `presence_engine.py` additions summary

You need these exact changes:

Add import:

Python Run

Add constant:

```
GOOD_MORNING_MESSAGES = [
    ...
```

Python Run

Add function:

```
def get_next_good_morning_message(employee):
```

Python Run

Replace function:

```
def send_good_morning_messages():
```

Python Run

---

14) Full copy-paste code blocks you need

A) `models.py` → add this

```
class GoodMorningLog(models.Model):
    employee = models.ForeignKey(
        "Employee",
        on_delete=models.CASCADE,
        related_name="good_morning_logs"
    )
    message = models.TextField()
    sent_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-sent_at"]
        verbose_name = "Good Morning Log"
        verbose_name_plural = "Good Morning Logs"

    def __str__(self):
        return f"{self.employee.display_name} - {self.sent_at:%Y-%m-%d %H:%M}"
```

B) `admin.py` → full updated version

```
from django.contrib import admin
from core.models import Employee, PresenceLog, ActivityTracker, ReminderLog, GoodMorningLog
```

```python
@admin.register(Employee)
class EmployeeAdmin(admin.ModelAdmin):
    list_display = (
        "display_name",
        "email",
        "is_active",
        "reminder_enabled",
        "reminder_interval_minutes",
        "last_good_morning_sent",
    )
    search_fields = ("display_name", "email")
    list_filter = ("is_active", "reminder_enabled")

    def last_good_morning_sent(self, obj):
        last_log = obj.good_morning_logs.order_by("-sent_at").first()
        return last_log.sent_at if last_log else "-"
    last_good_morning_sent.short_description = "Last Good Morning"


@admin.register(PresenceLog)
class PresenceLogAdmin(admin.ModelAdmin):
    list_display = ("employee", "status", "checked_at")
    search_fields = ("employee__display_name",)
    list_filter = ("status", "checked_at")


@admin.register(ActivityTracker)
class ActivityTrackerAdmin(admin.ModelAdmin):
    list_display = ("employee", "consecutive_available_count", "last_status")
    search_fields = ("employee__display_name",)


@admin.register(ReminderLog)
```

```python
class ReminderLogAdmin(admin.ModelAdmin):
    list_display = ("employee", "sent_at", "message")
    search_fields = ("employee__display_name", "message")
    list_filter = ("sent_at",)


@admin.register(GoodMorningLog)
class GoodMorningLogAdmin(admin.ModelAdmin):
    list_display = ("employee", "sent_at", "message")
    search_fields = ("employee__display_name", "message")
    list_filter = ("sent_at",)
```

C) `presence_engine.py` → add these parts

Add import:

Python Run

Add below `WELLNESS_REMINDER_MESSAGES`:

```python
GOOD_MORNING_MESSAGES = [
    "🌞 Good morning! Wishing you a productive and healthy workday. I'll gently remind you to tak
    "🌞 Good morning! Hope you start your day with energy, focus, and a healthy routine.",
    "🌸 Good morning! Wishing you a calm mind, good posture, and a productive day ahead.",
    "💧 Good morning! Stay hydrated, take mindful breaks, and have a wonderful workday.",
```

```
    "🌿 Good morning! A fresh start for a fresh day — work well and remember to take care of your
    "✨ Good morning! Small healthy habits today can make a big difference — have a great day ahe
    "🧘 Good morning! Begin your day with focus and balance. I'll be here to remind you to take s
    "🚶 Good morning! Wishing you a positive day filled with productivity and healthy movement.",
    "🌸 Good morning! Stay active, stretch often, and make today a great one.",
    "🌈 Good morning! Hope your day is productive, peaceful, and full of positive energy.",
    "💼 Good morning! Let's make today productive and healthy — don't forget to stretch and hydra
    "😊 Good morning! Wishing you a smooth workday with good focus, good health, and good energy.
]
```

Add helper function:

```
def get_next_good_morning_message(employee):
    """
    Return a good morning message that has not yet been used today for this employee.
    If already sent today, return None (only one Good Morning per day).
    If all messages were used historically, rotate safely.
    """
    today = timezone.localdate()

    already_sent_today = GoodMorningLog.objects.filter(
        employee=employee,
        sent_at__date=today
    ).exists()

    if already_sent_today:
        return None

    used_messages = set(
```

```
        GoodMorningLog.objects.filter(
            employee=employee
        ).values_list("message", flat=True)
    )

    for message in GOOD_MORNING_MESSAGES:
        if message not in used_messages:
            return message

    total_sent = GoodMorningLog.objects.filter(employee=employee).count()
    return GOOD_MORNING_MESSAGES[total_sent % len(GOOD_MORNING_MESSAGES)]
```

Replace `send_good_morning_messages()` بالكامل with:

```
def send_good_morning_messages():
    """
    Sends Good Morning message only to active employees who:
    - have initialized Teams chat
    - are currently online/working (not offline/away)
    - have NOT already received Good Morning today

    Runs at scheduled time Mon-Fri.
    """
    now = timezone.localtime()
    weekday = now.weekday()

    if weekday >= 5:
        print("Weekend. Skipping good morning messages.")
        return
```

```
    employees = Employee.objects.filter(
        is_active=True,
        reminder_enabled=True,
    )

    for employee in employees:
        if not has_teams_chat_initialized(employee):
            print(f"Skipping good morning for {employee.display_name}: no Teams chat initialized"
            continue

        good_morning_message = get_next_good_morning_message(employee)
        if good_morning_message is None:
            print(f"Skipping good morning for {employee.display_name}: already sent today")
            continue

        # Fetch live presence before sending Good Morning
        status = fetch_employee_presence(employee, save_log=True)

        if status is None:
            print(f"Skipping good morning for {employee.display_name}: presence unavailable")
            continue

        if status not in GOOD_MORNING_ALLOWED_STATUSES:
            print(
                f"Skipping good morning for {employee.display_name}: "
                f"status is {status} (not eligible)"
            )
            continue

        try:
            async_to_sync(send_proactive_message)(employee, good_morning_message)
            print(f"Good morning message sent to {employee.display_name}")
            print(f"Good morning message: {good_morning_message}")

            GoodMorningLog.objects.create(
```

```
                employee=employee,
                message=good_morning_message
            )

            print(f"Good morning log created for {employee.display_name}")

        except Exception as e:
            print(f"Failed to send good morning to {employee.display_name}: {e}")
```

---

15) After code change — run these commands

```bash
python manage.py makemigrations
python manage.py migrate
```

If scheduler is active, restart Django after changes.

---

16) How to test properly (important)

Test 1: Create model + admin visible

- Open admin
- Check you can see:
  - `Good Morning Logs`

Test 2: Run function manually in Django shell

```bash
```

Then:

```python
from core.presence_engine import send_good_morning_messages
```

Now check:

- Teams message sent
- Admin → `Good Morning Logs`
- one row created

Test 3: Run again same day

Run again:

```python
```

Expected:

- should print:
  - `already sent today`

- no second GoodMorningLog
- no duplicate Teams message

---

17) My recommendation (best production-friendly version)

For your current project, I strongly recommend this final logic:

- `PresenceLog` → keep current behavior
- `ReminderLog` → for wellness reminders only
- `GoodMorningLog` → separate model (best)
- one Good Morning max per employee per day
- unique rotating message per employee
- log only after successful Teams send

This is **exactly the right architecture** for your Teams wellness bot.

---

18) Small note about your `reset_daily_trackers()`

You currently have:

```
updated = ActivityTracker.objects.all().update(
    consecutive_available_count=0,
    last_status=None,
    reminder_sent=False,
)
```

⚠️ Make sure your `ActivityTracker` model actually has:

Python Run

Because in your admin display, I only saw:

- `consecutive_available_count`
- `last_status`

If `reminder_sent` field does **not exist** in model, this line will crash.

If that field is removed already, then change to:

```
updated = ActivityTracker.objects.all().update(
    consecutive_available_count=0,
    last_status=None,
`
```

Python Run

This is important.

---

19) Best next step I can do for you

If you want, I can now give you the **exact full final files** ready to copy-paste:

1. `models.py` **full updated**
2. `admin.py` **full updated**
3. `presence_engine.py` **full updated**
4. `scheduler.py` **final clean version** (with correct APScheduler jobs for:
    - 7:55 reset

- 8:05 good morning
- every 2 mins presence check)

👉 If you want, reply with:

“give me full final files”

and I’ll give you **copy-paste ready production-style code** for all 4 files with no missing parts.

```
from asgiref.sync import async_to_sync
from django.utils import timezone

from core.models import Employee, PresenceLog, ActivityTracker,
ReminderLog, GoodMorningLog
from core.services.graph import get_user_presence
from core.services.teams_sender import send_proactive_message


WORKING_STATUSES = [
    "Available",
    "Busy",
    "InACall",
    "InAMeeting",
]

GOOD_MORNING_ALLOWED_STATUSES = [
    "Available",
```

```
    "Busy",
    "InACall",
    "InAMeeting",
]

CHECK_INTERVAL_MINUTES = 2

WELLNESS_REMINDER_MESSAGES = [
    "💧 Hydration check! Take a sip of water and relax your shoulders.",
    "🧘 Short break time! Stand up, stretch, and refresh your body for 2 minutes.",
    "👀 Give your eyes a break — look away from the screen for a few moments.",
    "🚶 Time to move a little! A short walk can help reset your focus.",
    "🙆 Stretch reminder! Roll your neck and shoulders gently to reduce stiffness.",
    "🌿 Pause for a moment — breathe deeply and reset your energy.",
    "🪑 Sitting too long? Stand up and loosen your back and legs for a minute.",
    "🚰 Wellness check! Drink water and take a mindful pause before continuing.",
    "⏰ Time for a quick wellness break! Please drink some water and stretch a little.",
    "🖥️ Screen break time — rest your eyes and blink a few times slowly.",
    "🧍 Quick posture check! Sit tall or stand up and stretch for a moment.",
    "🌬️ Take a deep breath — a short breathing break can improve focus.",
```

```
    "🚰 Friendly reminder: hydrate, stretch, and return refreshed.",
    "✨ Small break, big benefit — take 2 minutes to move and recharge.",
]

GOOD_MORNING_MESSAGES = [
    "🌞 Good morning! Wishing you a productive and healthy workday. I'll gently remind you to take wellness breaks during the day.",
    "🌻 Good morning! Hope you start your day with energy, focus, and a healthy routine.",
    "🌷 Good morning! Wishing you a calm mind, good posture, and a productive day ahead.",
    "💧 Good morning! Stay hydrated, take mindful breaks, and have a wonderful workday.",
    "🌿 Good morning! A fresh start for a fresh day — work well and remember to take care of yourself.",
    "✨ Good morning! Small healthy habits today can make a big difference — have a great day ahead.",
    "🧘 Good morning! Begin your day with focus and balance. I'll be here to remind you to take short wellness breaks.",
    "🚶 Good morning! Wishing you a positive day filled with productivity and healthy movement.",
    "🌺 Good morning! Stay active, stretch often, and make today a great one.",
    "🌈 Good morning! Hope your day is productive, peaceful, and full of positive energy.",
    "💼 Good morning! Let's make today productive and healthy — don't forget to stretch and hydrate.",
    "😊 Good morning! Wishing you a smooth workday with good
```

```
focus, good health, and good energy.",
]


def is_within_working_hours():
    """
    Monday-Friday, 8:00 AM to 7:00 PM (Asia/Kolkata)
    """
    now = timezone.localtime()
    weekday = now.weekday()  # Monday=0, Sunday=6
    current_hour = now.hour

    is_weekday = weekday < 5
    within_hours = 8 <= current_hour < 19

    return is_weekday and within_hours


def get_next_reminder_message(employee):
    """
    Return a reminder message that has not yet been used today for this
employee.
    If all messages have been used today, fallback to first message.
    """
    today = timezone.localdate()

    used_messages_today = set(
        ReminderLog.objects.filter(
            employee=employee,
```

```
            sent_at__date=today
        ).values_list("message", flat=True)
    )

    for message in WELLNESS_REMINDER_MESSAGES:
        if message not in used_messages_today:
            return message

    # Fallback if all messages used in same day (unlikely but safe)
    return WELLNESS_REMINDER_MESSAGES[0]

def get_next_good_morning_message(employee):
    """
    Return a good morning message that has not yet been used today
for this employee.
    If a message was already sent today, return None (to ensure only one
per day).
    If all messages were used historically and you want rotation, fallback
safely.
    """
    today = timezone.localdate()

    # If already sent today, do not send again
    already_sent_today = GoodMorningLog.objects.filter(
        employee=employee,
        sent_at__date=today
    ).exists()

    if already_sent_today:
```

```
        return None

    # Optional: avoid repeating recent messages by checking all
historical GM logs
    used_messages = set(
        GoodMorningLog.objects.filter(
            employee=employee
        ).values_list("message", flat=True)
    )

    for message in GOOD_MORNING_MESSAGES:
        if message not in used_messages:
            return message

    # If all messages have been used historically, rotate from beginning
    # You can also randomize later if you want
    total_sent =
GoodMorningLog.objects.filter(employee=employee).count()
    return GOOD_MORNING_MESSAGES[total_sent %
len(GOOD_MORNING_MESSAGES)]

def has_teams_chat_initialized(employee):
    """
    Check whether employee has opened the bot at least once and
conversation details exist.
    """
    return (
        bool(employee.teams_user_id)
        and bool(employee.teams_conversation_id)
```

```
        and bool(employee.teams_service_url)
    )


def fetch_employee_presence(employee, save_log=True):
    """
    Fetch employee presence safely.
    Returns status string or None if fetch fails.

    If save_log=True, also stores PresenceLog.
    """
    try:
        presence_data = get_user_presence(employee.azure_user_id)
        status = presence_data.get("availability")

        print(f"{employee.display_name} status: {status}")

        if save_log:
            PresenceLog.objects.create(
                employee=employee,
                status=status or "Unknown"
            )

        return status

    except Exception as e:
        print(f"Presence fetch failed for {employee.display_name}: {e}")
        return None
```

```
def check_employee_presence():
    """
    Runs every 2 minutes during working hours.
    Sends reminder after continuous working duration reaches employee
threshold.
    """
    if not is_within_working_hours():
        print("Outside working hours. Skipping presence check.")
        return

    employees = Employee.objects.filter(
        is_active=True,
        reminder_enabled=True,
    )

    for employee in employees:
        # User must have initialized chat at least once
        if not has_teams_chat_initialized(employee):
            print(f"Skipping {employee.display_name}: no Teams
conversation details yet")
            continue

        status = fetch_employee_presence(employee, save_log=True)

        if status is None:
            continue

        tracker, _ =
```

```
ActivityTracker.objects.get_or_create(employee=employee)

        if status in WORKING_STATUSES:
            tracker.consecutive_available_count += 1
            print(f"Working detected ->
{tracker.consecutive_available_count}")
        else:
            print(f"Resetting counter for {employee.display_name}")
            tracker.consecutive_available_count = 0

        # Example: 60 mins / 2 mins = 30 checks
        required_checks = max(
            1,
            employee.reminder_interval_minutes //
CHECK_INTERVAL_MINUTES
        )

        if tracker.consecutive_available_count >= required_checks:
            reminder_message = get_next_reminder_message(employee)

            ReminderLog.objects.create(
                employee=employee,
                message=reminder_message
            )

            print(f"Reminder triggered for {employee.display_name}")
            print(f"Reminder message: {reminder_message}")

            try:
```

```
            async_to_sync(send_proactive_message)(employee,
reminder_message)
            print(f"Reminder sent to Teams for
{employee.display_name}")
        except Exception as e:
            print(f"Failed to send Teams reminder to
{employee.display_name}: {e}")

        # Reset after reminder so next reminder comes after another
full interval
        tracker.consecutive_available_count = 0

    tracker.last_status = status
    tracker.save()


def send_good_morning_messages():
    """
    Sends Good Morning message only to active employees who:
    - have initialized Teams chat
    - are currently online/working (not offline/away)
    - have NOT already received Good Morning today

    Runs at scheduled time Mon-Fri.
    """
    now = timezone.localtime()
    weekday = now.weekday()

    if weekday >= 5:
```

```
        print("Weekend. Skipping good morning messages.")
        return

    employees = Employee.objects.filter(
        is_active=True,
        reminder_enabled=True,
    )

    for employee in employees:
        if not has_teams_chat_initialized(employee):
            print(f"Skipping good morning for {employee.display_name}: no
Teams chat initialized")
            continue

        # Prevent duplicate Good Morning in same day
        good_morning_message =
get_next_good_morning_message(employee)
        if good_morning_message is None:
            print(f"Skipping good morning for {employee.display_name}:
already sent today")
            continue

        # Fetch live presence before sending Good Morning
        status = fetch_employee_presence(employee, save_log=True)

        if status is None:
            print(f"Skipping good morning for {employee.display_name}:
presence unavailable")
            continue
```

```
        if status not in GOOD_MORNING_ALLOWED_STATUSES:
            print(
                f"Skipping good morning for {employee.display_name}: "
                f"status is {status} (not eligible)"
            )
            continue

        try:
            async_to_sync(send_proactive_message)(employee,
good_morning_message)
            print(f"Good morning message sent to
{employee.display_name}")
            print(f"Good morning message: {good_morning_message}")

            # Create log only after successful send
            GoodMorningLog.objects.create(
                employee=employee,
                message=good_morning_message
            )

            print(f"Good morning log created for
{employee.display_name}")

        except Exception as e:
            print(f"Failed to send good morning to
{employee.display_name}: {e}")
```

```
def reset_daily_trackers():
    """
    Reset all activity counters before workday starts.
    Also delete all old PresenceLogs so only today's logs remain.
    Runs at 7:55 AM Mon-Fri.
    """
    # Delete old presence logs first
    today = timezone.localdate()
    deleted_count, _ = PresenceLog.objects.exclude(
        checked_at__date=today
    ).delete()

    # Reset trackers
    updated = ActivityTracker.objects.all().update(
        consecutive_available_count=0,
        last_status=None,
        reminder_sent=False,
    )

    print(f"Deleted old PresenceLog records: {deleted_count}")
    print(f"Reset {updated} activity trackers")
```

is this correct presense_engine.py

Yes — **your** `presence_engine.py` **is almost correct** ✅

But there are **2 important fixes** and **1 best-practice improvement** you should do before using it.

✅ Short answer

Your file is mostly correct, and your **Good Morning logging logic is implemented properly** 👍

But you need to fix:

1. **Bug in** `check_employee_presence()`
   You are creating `ReminderLog` **before** sending Teams message
   → should create log **only after successful send**
2. **Possible bug in** `reset_daily_trackers()`
   You are updating:

   Python Run

   If `ActivityTracker` model **does not have** `reminder_sent`, this will crash.
3. **Optional improvement**
   Add a blank line between functions (PEP8 style) and use `ceil` instead of floor division for reminder interval accuracy.

✅ Final verdict

Good Morning part:

✔ Correct

GoodMorningLog part:

✔ Correct

Unique Good Morning messages:

✔️ Correct

Admin will work (if model/admin updated):

✔️ Correct

Wellness reminder send flow:

⚠️ Needs one important fix

🔴 Important Fix #1 — `ReminderLog` should be created only after successful Teams send

Right now in your `check_employee_presence()`:

```
ReminderLog.objects.create(
    employee=employee,
    message=reminder_message
)

print(f"Reminder triggered for {employee.display_name}")
print(f"Reminder message: {reminder_message}")

try:
    async_to_sync(send_proactive_message)(employee, reminder_message)
    print(f"Reminder sent to Teams for {employee.display_name}")
except Exception as e:
    print(f"Failed to send Teams reminder to {employee.display_name}: {e}")
```

❌ Problem:

If Teams send fails, you still create `ReminderLog`.

That means admin will show:

- reminder was sent
- but actually Teams message failed

This is **incorrect logging**.

✅ Correct version

Replace that block inside `check_employee_presence()` with this:

```
print(f"Reminder triggered for {employee.display_name}")
print(f"Reminder message: {reminder_message}")

try:
    async_to_sync(send_proactive_message)(employee, reminder_message)
    print(f"Reminder sent to Teams for {employee.display_name}")

    ReminderLog.objects.create(
        employee=employee,
        message=reminder_message
    )

    print(f"Reminder log created for {employee.display_name}")

    # Reset after successful reminder so next reminder comes after another full interval
    tracker.consecutive_available_count = 0
```

```
except Exception as e:
    print(f"Failed to send Teams reminder to {employee.display_name}: {e}")
```

⚠️ Very important:

Because you moved reset inside `try`, also **remove this old line below the try/except**:

```python
updated = ActivityTracker.objects.all().update(
    consecutive_available_count=0,
    last_status=None,
    reminder_sent=False,
)
```

Otherwise it will still reset even when send fails.

🔴 Important Fix #2 — `reminder_sent=False` may crash

At bottom:

❌ Problem:

If your `ActivityTracker` model does **not** have field `reminder_sent`, Django will raise error like:

- `FieldDoesNotExist`
- or `Cannot resolve keyword 'reminder_sent'`

From the code you shared earlier, I suspect you may **not need this field anymore**.

✅ Safer version (recommended)

Change to:

```
updated = ActivityTracker.objects.all().update(
    consecutive_available_count=0,
    last_status=None,
`
```

✅ Best-practice improvement #3 — reminder interval calculation

Right now you use:

```
required_checks = max(
    1,
    employee.reminder_interval_minutes // CHECK_INTERVAL_MINUTES
`
```

This uses **floor division**.

Example:

- reminder interval = **5 min**
- check interval = **2 min**
- `5 // 2 = 2`

That means reminder comes after:

- 2 checks × 2 min = **4 minutes**, not 5

So it can trigger **early**.

✅ Better approach: use `math.ceil`

At top add:

```python
```

Then replace:

```python
required_checks = max(
    1,
    employee.reminder_interval_minutes // CHECK_INTERVAL_MINUTES
`
```

with:

```
```

```python
required_checks = max(
    1,
    math.ceil(employee.reminder_interval_minutes / CHECK_INTERVAL_MINUTES)
`
```

Python Run

Example:

- 5 min / 2 min = 2.5 → ceil = 3
- 3 checks × 2 min = **6 minutes**
- safer than sending early

👉 This is the **correct production logic**

✅ So is your `send_good_morning_messages()` correct?

Yes — your Good Morning logic is good 👌

It correctly does:

- checks weekday
- checks employee active + reminder enabled
- checks Teams chat initialized
- prevents duplicate same day
- fetches live presence
- only allows certain statuses
- sends message
- creates `GoodMorningLog` **after successful send**

This part is **properly implemented**.

✅ Final corrected `check_employee_presence()` (copy-paste)

Use this full version:

```
import math


def check_employee_presence():
    """
    Runs every 2 minutes during working hours.
    Sends reminder after continuous working duration reaches employee threshold.
    """
    if not is_within_working_hours():
        print("Outside working hours. Skipping presence check.")
        return

    employees = Employee.objects.filter(
        is_active=True,
        reminder_enabled=True,
    )

    for employee in employees:
        # User must have initialized chat at least once
        if not has_teams_chat_initialized(employee):
            print(f"Skipping {employee.display_name}: no Teams conversation details yet")
            continue

        status = fetch_employee_presence(employee, save_log=True)

        if status is None:
            continue

        tracker, _ = ActivityTracker.objects.get_or_create(employee=employee)
```

```
        if status in WORKING_STATUSES:
            tracker.consecutive_available_count += 1
            print(f"Working detected -> {employee.display_name}: {tracker.consecutive_available_c
        else:
            print(f"Resetting counter for {employee.display_name}")
            tracker.consecutive_available_count = 0

        # Example: 60 mins / 2 mins = 30 checks
        required_checks = max(
            1,
            math.ceil(employee.reminder_interval_minutes / CHECK_INTERVAL_MINUTES)
        )

        if tracker.consecutive_available_count >= required_checks:
            reminder_message = get_next_reminder_message(employee)

            print(f"Reminder triggered for {employee.display_name}")
            print(f"Reminder message: {reminder_message}")

            try:
                async_to_sync(send_proactive_message)(employee, reminder_message)
                print(f"Reminder sent to Teams for {employee.display_name}")

                ReminderLog.objects.create(
                    employee=employee,
                    message=reminder_message
                )

                print(f"Reminder log created for {employee.display_name}")

                # Reset only after successful send
                tracker.consecutive_available_count = 0

            except Exception as e:
```

```
                print(f"Failed to send Teams reminder to {employee.display_name}: {e}")

        tracker.last_status = status
        tracker.save()
```

✅ Final corrected `reset_daily_trackers()` (copy-paste)

Use this safer version:

```
def reset_daily_trackers():
    """
    Reset all activity counters before workday starts.
    Also delete all old PresenceLogs so only today's logs remain.
    Runs at 7:55 AM Mon-Fri.
    """
    # Delete old presence logs first
    today = timezone.localdate()
    deleted_count, _ = PresenceLog.objects.exclude(
        checked_at__date=today
    ).delete()

    # Reset trackers
    updated = ActivityTracker.objects.all().update(
        consecutive_available_count=0,
        last_status=None,
    )

    print(f"Deleted old PresenceLog records: {deleted_count}")
    print(f"Reset {updated} activity trackers")
```

✅ Final status of your file

Your current `presence_engine.py` is:

- **85–90% correct**
- Good Morning feature is **implemented correctly**
- Just fix:
  - reminder log creation position
  - possible `reminder_sent` field issue
  - optionally use `math.ceil`

✅ My recommended final imports at top

Your imports should ideally be:

```
import math

from asgiref.sync import async_to_sync
from django.utils import timezone

from core.models import Employee, PresenceLog, ActivityTracker, ReminderLog, GoodMorningLog
from core.services.graph import get_user_presence
from core.services.teams_sender import send_proactive_message
```

🚀 Best next step (recommended now)

Now the **next thing you should do immediately** is:

1. Confirm `GoodMorningLog` model exists in `models.py`
2. Run:

```bash
python manage.py makemigrations
```

3. Update `admin.py`
4. Restart Django server
5. Test from shell:

```bash
```

Then:

```python
from core.presence_engine import send_good_morning_messages
```

🔥 Best help I can give you next

If you want, I can now give you the **exact final copy-paste version of all 3 files**:

- `models.py`
- `admin.py`
- `presence_engine.py`

with **all corrections already applied** (including the reminder log fix and safe reset).

👉 Just reply:

"give me final copy-paste files"

and I'll send you the **fully corrected production-ready code** in one go.

can you give me fully updated codefile

Yes — absolutely.
Below I'm giving you the **fully updated final copy-paste code** for:

1. `core/models.py`
2. `core/admin.py`
3. `core/presence_engine.py`

This version includes:

- ✅ `GoodMorningLog` model
- ✅ Admin support for `GoodMorningLog`
- ✅ 12 unique Good Morning messages
- ✅ One Good Morning per employee per day

- ✅ Different Good Morning messages per employee over time
- ✅ `ReminderLog` created **only after successful Teams send**
- ✅ `GoodMorningLog` created **only after successful Teams send**
- ✅ Safer `math.ceil()` interval logic
- ✅ Safe `reset_daily_trackers()` (no `reminder_sent` issue)

---

1) `core/models.py` (FULL UPDATED FILE)

> Replace your current `core/models.py` with this full version **only if your existing models are same/similar**.
> If you already have extra fields, keep them and only merge carefully.

```
from django.db import models


class Employee(models.Model):
    display_name = models.CharField(max_length=255)
    email = models.EmailField(unique=True)
    azure_user_id = models.CharField(max_length=255, unique=True)

    # Teams conversation details for proactive messaging
    teams_user_id = models.CharField(max_length=255, blank=True, null=True)
    teams_conversation_id = models.CharField(max_length=255, blank=True, null=True)
    teams_service_url = models.TextField(blank=True, null=True)

    is_active = models.BooleanField(default=True)
    reminder_enabled = models.BooleanField(default=True)
    reminder_interval_minutes = models.PositiveIntegerField(default=60)
```

```
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["display_name"]

    def __str__(self):
        return f"{self.display_name} ({self.email})"


class PresenceLog(models.Model):
    employee = models.ForeignKey(
        Employee,
        on_delete=models.CASCADE,
        related_name="presence_logs"
    )
    status = models.CharField(max_length=100)
    checked_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-checked_at"]

    def __str__(self):
        return f"{self.employee.display_name} - {self.status} - {self.checked_at:%Y-%m-%d %H:%M}"


class ActivityTracker(models.Model):
    employee = models.OneToOneField(
        Employee,
        on_delete=models.CASCADE,
        related_name="activity_tracker"
    )
    consecutive_available_count = models.PositiveIntegerField(default=0)
    last_status = models.CharField(max_length=100, blank=True, null=True)
```

```
    def __str__(self):
        return f"{self.employee.display_name} - {self.consecutive_available_count}"


class ReminderLog(models.Model):
    employee = models.ForeignKey(
        Employee,
        on_delete=models.CASCADE,
        related_name="reminder_logs"
    )
    message = models.TextField()
    sent_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-sent_at"]
        verbose_name = "Reminder Log"
        verbose_name_plural = "Reminder Logs"

    def __str__(self):
        return f"{self.employee.display_name} - {self.sent_at:%Y-%m-%d %H:%M}"


class GoodMorningLog(models.Model):
    employee = models.ForeignKey(
        Employee,
        on_delete=models.CASCADE,
        related_name="good_morning_logs"
    )
    message = models.TextField()
    sent_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-sent_at"]
        verbose_name = "Good Morning Log"
        verbose_name_plural = "Good Morning Logs"
```

```
    def __str__(self):
        return f"{self.employee.display_name} - {self.sent_at:%Y-%m-%d %H:%M}"
```

---

2) `core/admin.py` (FULL UPDATED FILE)

> Replace your current `core/admin.py` with this:

```
from django.contrib import admin
from core.models import Employee, PresenceLog, ActivityTracker, ReminderLog, GoodMorningLog


@admin.register(Employee)
class EmployeeAdmin(admin.ModelAdmin):
    list_display = (
        "display_name",
        "email",
        "is_active",
        "reminder_enabled",
        "reminder_interval_minutes",
        "last_good_morning_sent",
    )
    search_fields = ("display_name", "email")
    list_filter = ("is_active", "reminder_enabled")

    def last_good_morning_sent(self, obj):
        last_log = obj.good_morning_logs.order_by("-sent_at").first()
        return last_log.sent_at if last_log else "-"
```

```
    last_good_morning_sent.short_description = "Last Good Morning"


@admin.register(PresenceLog)
class PresenceLogAdmin(admin.ModelAdmin):
    list_display = ("employee", "status", "checked_at")
    search_fields = ("employee__display_name",)
    list_filter = ("status", "checked_at")


@admin.register(ActivityTracker)
class ActivityTrackerAdmin(admin.ModelAdmin):
    list_display = ("employee", "consecutive_available_count", "last_status")
    search_fields = ("employee__display_name",)


@admin.register(ReminderLog)
class ReminderLogAdmin(admin.ModelAdmin):
    list_display = ("employee", "sent_at", "message")
    search_fields = ("employee__display_name", "message")
    list_filter = ("sent_at",)


@admin.register(GoodMorningLog)
class GoodMorningLogAdmin(admin.ModelAdmin):
    list_display = ("employee", "sent_at", "message")
    search_fields = ("employee__display_name", "message")
    list_filter = ("sent_at",)
```

---

3) `core/presence_engine.py` (FULL UPDATED FILE)

> Replace your current `core/presence_engine.py` with this full final version:

```python
import math

from asgiref.sync import async_to_sync
from django.utils import timezone

from core.models import Employee, PresenceLog, ActivityTracker, ReminderLog, GoodMorningLog
from core.services.graph import get_user_presence
from core.services.teams_sender import send_proactive_message


WORKING_STATUSES = [
    "Available",
    "Busy",
    "InACall",
    "InAMeeting",
]

GOOD_MORNING_ALLOWED_STATUSES = [
    "Available",
    "Busy",
    "InACall",
    "InAMeeting",
]

CHECK_INTERVAL_MINUTES = 2

WELLNESS_REMINDER_MESSAGES = [
    "💧 Hydration check! Take a sip of water and relax your shoulders.",
    "🧘 Short break time! Stand up, stretch, and refresh your body for 2 minutes.",
    "👀 Give your eyes a break — look away from the screen for a few moments.",
    "🚶 Time to move a little! A short walk can help reset your focus.",
    "🧘 Stretch reminder! Roll your neck and shoulders gently to reduce stiffness.",
    "🌿 Pause for a moment — breathe deeply and reset your energy.",
```

```
    "🪑 Sitting too long? Stand up and loosen your back and legs for a minute.",
    "🚰 Wellness check! Drink water and take a mindful pause before continuing.",
    "⏰ Time for a quick wellness break! Please drink some water and stretch a little.",
    "🖥️ Screen break time — rest your eyes and blink a few times slowly.",
    "🧍 Quick posture check! Sit tall or stand up and stretch for a moment.",
    "🌬️ Take a deep breath — a short breathing break can improve focus.",
    "🚰 Friendly reminder: hydrate, stretch, and return refreshed.",
    "✨ Small break, big benefit — take 2 minutes to move and recharge.",
]

GOOD_MORNING_MESSAGES = [
    "🌞 Good morning! Wishing you a productive and healthy workday. I'll gently remind you to tak
    "🌻 Good morning! Hope you start your day with energy, focus, and a healthy routine.",
    "🌸 Good morning! Wishing you a calm mind, good posture, and a productive day ahead.",
    "💧 Good morning! Stay hydrated, take mindful breaks, and have a wonderful workday.",
    "🌿 Good morning! A fresh start for a fresh day — work well and remember to take care of your
    "✨ Good morning! Small healthy habits today can make a big difference — have a great day ahe
    "🧘 Good morning! Begin your day with focus and balance. I'll be here to remind you to take s
    "🚶 Good morning! Wishing you a positive day filled with productivity and healthy movement.",
    "🌺 Good morning! Stay active, stretch often, and make today a great one.",
    "🌈 Good morning! Hope your day is productive, peaceful, and full of positive energy.",
    "💼 Good morning! Let's make today productive and healthy — don't forget to stretch and hydra
    "😊 Good morning! Wishing you a smooth workday with good focus, good health, and good energy.
]


def is_within_working_hours():
    """
    Monday-Friday, 8:00 AM to 7:00 PM (Asia/Kolkata)
    """
    now = timezone.localtime()
    weekday = now.weekday()  # Monday=0, Sunday=6
    current_hour = now.hour

    is_weekday = weekday < 5
```

```
    within_hours = 8 <= current_hour < 19

    return is_weekday and within_hours


def get_next_reminder_message(employee):
    """
    Return a reminder message that has not yet been used today for this employee.
    If all messages have been used today, fallback to first message.
    """
    today = timezone.localdate()

    used_messages_today = set(
        ReminderLog.objects.filter(
            employee=employee,
            sent_at__date=today
        ).values_list("message", flat=True)
    )

    for message in WELLNESS_REMINDER_MESSAGES:
        if message not in used_messages_today:
            return message

    # Fallback if all messages used in same day (unlikely but safe)
    return WELLNESS_REMINDER_MESSAGES[0]


def get_next_good_morning_message(employee):
    """
    Return a Good Morning message for this employee.

    Rules:
    - Only one Good Morning per employee per day
    - Prefer a message that employee has not received before
    - If all messages are already used historically, rotate safely
```

```
    """
    today = timezone.localdate()

    # Prevent duplicate Good Morning on same day
    already_sent_today = GoodMorningLog.objects.filter(
        employee=employee,
        sent_at__date=today
    ).exists()

    if already_sent_today:
        return None

    # Try to avoid repeating historical messages until all are used
    used_messages = set(
        GoodMorningLog.objects.filter(
            employee=employee
        ).values_list("message", flat=True)
    )

    for message in GOOD_MORNING_MESSAGES:
        if message not in used_messages:
            return message

    # If all messages already used historically, rotate from beginning
    total_sent = GoodMorningLog.objects.filter(employee=employee).count()
    return GOOD_MORNING_MESSAGES[total_sent % len(GOOD_MORNING_MESSAGES)]


def has_teams_chat_initialized(employee):
    """
    Check whether employee has opened the bot at least once and conversation details exist.
    """
    return (
        bool(employee.teams_user_id)
        and bool(employee.teams_conversation_id)
```

```
        and bool(employee.teams_service_url)
    )


def fetch_employee_presence(employee, save_log=True):
    """
    Fetch employee presence safely.
    Returns status string or None if fetch fails.

    If save_log=True, also stores PresenceLog.
    """
    try:
        presence_data = get_user_presence(employee.azure_user_id)
        status = presence_data.get("availability")

        print(f"{employee.display_name} status: {status}")

        if save_log:
            PresenceLog.objects.create(
                employee=employee,
                status=status or "Unknown"
            )

        return status

    except Exception as e:
        print(f"Presence fetch failed for {employee.display_name}: {e}")
        return None


def check_employee_presence():
    """
    Runs every 2 minutes during working hours.
    Sends reminder after continuous working duration reaches employee threshold.
    """
```

```
    if not is_within_working_hours():
        print("Outside working hours. Skipping presence check.")
        return

    employees = Employee.objects.filter(
        is_active=True,
        reminder_enabled=True,
    )

    for employee in employees:
        # User must have initialized chat at least once
        if not has_teams_chat_initialized(employee):
            print(f"Skipping {employee.display_name}: no Teams conversation details yet")
            continue

        status = fetch_employee_presence(employee, save_log=True)

        if status is None:
            continue

        tracker, _ = ActivityTracker.objects.get_or_create(employee=employee)

        if status in WORKING_STATUSES:
            tracker.consecutive_available_count += 1
            print(f"Working detected -> {employee.display_name}: {tracker.consecutive_available_c
        else:
            print(f"Resetting counter for {employee.display_name}")
            tracker.consecutive_available_count = 0

        # Example:
        # 60 mins / 2 mins = 30 checks
        # Use ceil so reminders are never sent earlier than configured interval
        required_checks = max(
            1,
            math.ceil(employee.reminder_interval_minutes / CHECK_INTERVAL_MINUTES)
```

```
        )

        if tracker.consecutive_available_count >= required_checks:
            reminder_message = get_next_reminder_message(employee)

            print(f"Reminder triggered for {employee.display_name}")
            print(f"Reminder message: {reminder_message}")

            try:
                async_to_sync(send_proactive_message)(employee, reminder_message)
                print(f"Reminder sent to Teams for {employee.display_name}")

                # Create log only after successful send
                ReminderLog.objects.create(
                    employee=employee,
                    message=reminder_message
                )
                print(f"Reminder log created for {employee.display_name}")

                # Reset only after successful send
                tracker.consecutive_available_count = 0

            except Exception as e:
                print(f"Failed to send Teams reminder to {employee.display_name}: {e}")

        tracker.last_status = status
        tracker.save()


def send_good_morning_messages():
    """
    Sends Good Morning message only to active employees who:
    - have initialized Teams chat
    - are currently online/working (not offline/away)
    - have NOT already received Good Morning today
```

```
    Runs at scheduled time Mon-Fri.
    """
    now = timezone.localtime()
    weekday = now.weekday()

    if weekday >= 5:
        print("Weekend. Skipping good morning messages.")
        return

    employees = Employee.objects.filter(
        is_active=True,
        reminder_enabled=True,
    )

    for employee in employees:
        if not has_teams_chat_initialized(employee):
            print(f"Skipping good morning for {employee.display_name}: no Teams chat initialized"
            continue

        # Prevent duplicate Good Morning in same day
        good_morning_message = get_next_good_morning_message(employee)
        if good_morning_message is None:
            print(f"Skipping good morning for {employee.display_name}: already sent today")
            continue

        # Fetch live presence before sending Good Morning
        status = fetch_employee_presence(employee, save_log=True)

        if status is None:
            print(f"Skipping good morning for {employee.display_name}: presence unavailable")
            continue

        if status not in GOOD_MORNING_ALLOWED_STATUSES:
            print(
```

```
                f"Skipping good morning for {employee.display_name}: "
                f"status is {status} (not eligible)"
            )
            continue

        try:
            async_to_sync(send_proactive_message)(employee, good_morning_message)
            print(f"Good morning message sent to {employee.display_name}")
            print(f"Good morning message: {good_morning_message}")

            # Create log only after successful send
            GoodMorningLog.objects.create(
                employee=employee,
                message=good_morning_message
            )
            print(f"Good morning log created for {employee.display_name}")

        except Exception as e:
            print(f"Failed to send good morning to {employee.display_name}: {e}")


def reset_daily_trackers():
    """
    Reset all activity counters before workday starts.
    Also delete all old PresenceLogs so only today's logs remain.
    Runs at 7:55 AM Mon-Fri.
    """
    # Delete old presence logs first
    today = timezone.localdate()
    deleted_count, _ = PresenceLog.objects.exclude(
        checked_at__date=today
    ).delete()

    # Reset trackers
    updated = ActivityTracker.objects.all().update(
```

```
        consecutive_available_count=0,
        last_status=None,
    )

    print(f"Deleted old PresenceLog records: {deleted_count}")
    print(f"Reset {updated} activity trackers")
```

---

4) VERY IMPORTANT — after updating models

Because we added `GoodMorningLog`, now run:

```bash
python manage.py makemigrations
```

---

5) Test steps (important)

After code update:

Run server again

```bash
```

Test Good Morning manually in shell

```bash
```

Then:

```python
from core.presence_engine import send_good_morning_messages
```

Check:

- Teams message sent
- `GoodMorningLog` created
- Admin shows `Good Morning Logs`

Test reminder manually

```python
from core.presence_engine import check_employee_presence
```

Check:

- if threshold met → reminder sent
- `ReminderLog` created **only after success**

6) One important note about your existing `models.py`

⚠️ If your current `models.py` already has:

- extra fields
- custom Meta
- custom indexes
- additional methods

then **do not blindly overwrite the whole file**.

In that case:

- keep your existing file
- only add:
  - `GoodMorningLog`
- ensure `ActivityTracker` has:
  - `consecutive_available_count`
  - `last_status`

If you want, I can also do the **safest version** next:

👉 You paste your **current full** `models.py`, and I'll return:

- **exact merged final** `models.py`
- no risk of losing your existing fields

That would be the best/safer option for your project.

7) Best next step for you now

I recommend next I give you the **final** `scheduler.py` also, so your jobs run cleanly:

- **7:55 AM** → `reset_daily_trackers()`
- **8:05 AM** → `send_good_morning_messages()`
- **Every 2 mins (8 AM–7 PM Mon–Fri)** → `check_employee_presence()`

👉 If you want that, reply:

"give me final scheduler.py"

and I'll send you the **production-safe APScheduler file** matching this code exactly.

now you see we delete presense logs everyday morning just like that i want to delete good morning logs every week every monday morning can we do that okay no weekly can we delete logs everyday morning just like presense logs

Yes — **absolutely, you can do that** ✅
And for your current project, **daily deletion is actually the better/simple option**.

Since you said:

- ❌ not weekly
- ✅ delete **GoodMorningLog every day morning**
- same like `PresenceLog`

That is **totally fine** and easy to implement.

✅ What you want now

Every morning in `reset_daily_trackers()`:

- delete old `PresenceLog` → already happening
- also delete old `GoodMorningLog` → **keep only today's good morning logs**

So after reset:

- `PresenceLog` = only today's records
- `GoodMorningLog` = only today's records
- `ReminderLog` = still keep history (recommended)

⚠️ Important effect of deleting `GoodMorningLog` daily

Right now your function `get_next_good_morning_message()` uses **historical** `GoodMorningLog` **records** to avoid repeating messages over many days.

If you delete `GoodMorningLog` daily, then:

- yesterday's Good Morning logs are gone
- system **cannot remember which GM messages were used before**
- result:
  - it will always choose the **first unused message today**
  - since no old logs exist, it will usually send the **first message again every day**

👉 So if you delete `GoodMorningLog` daily, your current "different message across days" logic will **stop working**.

✅ Best solution (simple and correct)

If you want to **delete GoodMorningLog daily**, then also change the Good Morning selection logic to:

- choose message based on **today's date**
- or choose message based on **employee id + today**
- so message still changes daily even without historical logs

This is the best approach for your new requirement.

✅ Recommended approach now

We will do **2 changes**:

1. Update `reset_daily_trackers()`
   → also delete old `GoodMorningLog`
2. Update `get_next_good_morning_message(employee)`
   → **do not depend on old GoodMorningLog history**
   → use a **rotation formula based on date + employee.id**

This way:

- only **1 GoodMorningLog per day**
- logs deleted every morning
- still **different Good Morning messages**
- different employees can get different messages

- no need to keep historical GM logs

✅ Change #1 — Replace `get_next_good_morning_message()` with this new version

Replace your current function with:

```
def get_next_good_morning_message(employee):
    """
    Return a Good Morning message for this employee.

    Rules:
    - Only one Good Morning per employee per day
    - Since old GoodMorningLogs are deleted daily, do NOT rely on history
    - Rotate message using today's date + employee id
    """
    today = timezone.localdate()

    # Prevent duplicate Good Morning on same day
    already_sent_today = GoodMorningLog.objects.filter(
        employee=employee,
        sent_at__date=today
    ).exists()

    if already_sent_today:
        return None

    # Rotate messages daily using date + employee id
    # This gives variation across days and across employees
    day_number = today.toordinal()
```

```
    employee_seed = employee.id or 0

    index = (day_number + employee_seed) % len(GOOD_MORNING_MESSAGES)
    return GOOD_MORNING_MESSAGES[index]
```

✅ Why this is best now

Because after daily deletion, no old GM logs exist.

So instead of using DB history, this logic uses:

- **today's date**
- **employee id**

That means:

- message changes day by day
- different employees often get different messages
- no historical table needed
- still one message per day only

✅ Change #2 — Update `reset_daily_trackers()` to also delete old GoodMorningLog

Replace your current `reset_daily_trackers()` with this:

```python
def reset_daily_trackers():
    """
    Reset all activity counters before workday starts.
    Also delete all old PresenceLogs and GoodMorningLogs
    so only today's logs remain.
    Runs at 7:55 AM Mon-Fri.
    """
    today = timezone.localdate()

    # Delete old presence logs
    deleted_presence_count, _ = PresenceLog.objects.exclude(
        checked_at__date=today
    ).delete()

    # Delete old good morning logs
    deleted_good_morning_count, _ = GoodMorningLog.objects.exclude(
        sent_at__date=today
    ).delete()

    # Reset trackers
    updated = ActivityTracker.objects.all().update(
        consecutive_available_count=0,
        last_status=None,
    )

    print(f"Deleted old PresenceLog records: {deleted_presence_count}")
    print(f"Deleted old GoodMorningLog records: {deleted_good_morning_count}")
    print(f"Reset {updated} activity trackers")
```

✅ Final behavior after this change

Every morning at reset time:

- delete all old `PresenceLog` except today
- delete all old `GoodMorningLog` except today
- reset `ActivityTracker`

Then later when `send_good_morning_messages()` runs:

- if employee already got GM today → skip
- otherwise:
  - calculate message using **date + employee.id**
  - send it
  - create `GoodMorningLog`

✅ This is exactly what you should do now

So your **final changes are only these 2 functions**:

---

1) Final `get_next_good_morning_message()`

```
def get_next_good_morning_message(employee):
    """
    Return a Good Morning message for this employee.

    Rules:
    - Only one Good Morning per employee per day
    - Since old GoodMorningLogs are deleted daily, do NOT rely on history
    - Rotate message using today's date + employee id
    """
```

```
    today = timezone.localdate()

    # Prevent duplicate Good Morning on same day
    already_sent_today = GoodMorningLog.objects.filter(
        employee=employee,
        sent_at__date=today
    ).exists()

    if already_sent_today:
        return None

    # Rotate messages daily using date + employee id
    # This gives variation across days and across employees
    day_number = today.toordinal()
    employee_seed = employee.id or 0

    index = (day_number + employee_seed) % len(GOOD_MORNING_MESSAGES)
    return GOOD_MORNING_MESSAGES[index]
```

---

2) Final `reset_daily_trackers()`

```
def reset_daily_trackers():
    """
    Reset all activity counters before workday starts.
    Also delete all old PresenceLogs and GoodMorningLogs
    so only today's logs remain.
    Runs at 7:55 AM Mon-Fri.
    """
    today = timezone.localdate()
```

```
    # Delete old presence logs
    deleted_presence_count, _ = PresenceLog.objects.exclude(
        checked_at__date=today
    ).delete()

    # Delete old good morning logs
    deleted_good_morning_count, _ = GoodMorningLog.objects.exclude(
        sent_at__date=today
    ).delete()

    # Reset trackers
    updated = ActivityTracker.objects.all().update(
        consecutive_available_count=0,
        last_status=None,
    )

    print(f"Deleted old PresenceLog records: {deleted_presence_count}")
    print(f"Deleted old GoodMorningLog records: {deleted_good_morning_count}")
    print(f"Reset {updated} activity trackers")
```

✅ My recommendation (very important)

For your project, I recommend this final policy:

- `PresenceLog` → delete daily ✅
- `GoodMorningLog` → delete daily ✅
- `ReminderLog` → **do NOT delete daily** (keep history) ✅

Why?

Because:

- Presence logs grow fast
- Good morning logs are mostly just daily status proof
- Reminder logs are more valuable for tracking actual wellness reminders over time

---

🚀 Best next step

If you want, I can now give you the **fully updated final** `presence_engine.py` **again** with these new changes already included (daily GoodMorningLog deletion + new rotating logic), so you can **copy-paste once and finish**.

👉 Just reply:

“give me final presence_engine.py with daily GM deletion”

and I’ll send the exact final full file.